

RadixAttention, Worked Out by Hand

PREFIX SHARING — WHY EVERY REQUEST WITH THE SAME SYSTEM PROMPT SHOULD PAY ONCE

What this document does. It takes the core idea of RadixAttention (Zheng et al., SGLang, 2024) — keep an automatic trie of token sequences seen so far, and reuse the KV cache of any already-computed prefix — and works the arithmetic of *how much it saves*. In production, requests are not independent: a chatbot ships the same 2100-token system prompt on every call; a RAG service shares document chunks; a code assistant shares instruction templates. Recomputing that shared prefix’s KV for each request is pure waste: identical input, identical output, identical bytes. We quantify the waste, and the saving, on a small concrete request set. Every figure is produced by a reference script (Appendix A, `m04_t03_engine.py`).

The one idea. A shared prefix should be computed *once* and read *many* times. The theoretical ceiling on how often that helps is a property of your *prompts*, not your engine: $\text{max_hit_rate} = (N - \text{unique_prefixes})/N$.

CONCEPTS COVERED, IN ORDER

the prefix problem PagedAttention does not solve; the radix-trie model; the hit-rate ceiling formula; a worked 8-request set landing at 62.5%; KV tokens recomputed with and without sharing; the “system prompt paid once” headline; the byte-stability footgun; and the production-scale 98.6% ceiling.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The problem PagedAttention does not solve	2
2	Step 1 — The hit-rate ceiling	2
3	Step 2 — A worked example: 8 requests, 3 distinct prefixes	3
4	Step 3 — Tokens recomputed, with and without sharing	3
5	Step 4 — The “system prompt paid once” headline	3
6	Step 5 — The byte-stability footgun	4
7	Step 6 — The production-scale ceiling	4
8	The argument, at a glance	5
8.1	Equation cheat-sheet	5

Appendix A — Reference implementation

5

1 The problem PagedAttention does not solve

PagedAttention (Topic 2) solves *fragmentation*: within one request, use HBM efficiently. It does not, on its own, solve *sharing*: across requests with a common prefix, compute once and read many times. That requires a second layer — prefix caching.

Intuition

RadixAttention builds an in-memory *radix trie* keyed on the sequence of token IDs. When a request arrives, its tokens are matched against the trie. The longest common prefix is *already in HBM* from earlier requests, so we point the new request's block table at those existing KV blocks and only compute KV for the new *suffix*. Reference counting keeps a shared node alive while any in-flight request depends on it; an LRU policy evicts cold leaves under pressure. Because paging already lets two block tables point at the same physical block, prefix sharing is "free" to express — it is just a second request reusing the first request's blocks.

Quantity	Symbol	Value here
KV per token (FP16, GQA-8, 80L)	κ	320 KiB/token
Nexus system prompt	L_{sys}	2100 tokens
Requests in the worked set	N	8

The per-token rate $\kappa = 2 n_{\text{kv}} d_{\text{head}} b N_{\text{layers}} = 320 \text{ KiB}$ is carried straight over from Topic 1.

2 Step 1 — The hit-rate ceiling

The maximum hit rate is decided by your prompts, before any engine runs. Bucket each request by its shared prefix. A request can be a "hit" only if some *earlier* request already populated its bucket; the first request in each bucket must pay full price. So the number of unavoidable misses equals the number of *distinct* prefixes:

Prefix-cache hit-rate ceiling

$$\text{max_hit_rate} = \frac{N - \text{unique_prefixes}}{N}.$$

Watch out

This is a ceiling, not a guarantee. The engine can fall short of it — through eviction under memory pressure, requests landing on different replicas, or a tokenizer producing non-deterministic IDs. But it can never *exceed* it. The gap between the ceiling and the observed rate is the diagnostic: if the ceiling is 98.6% and you observe 85%, the problem is scheduler eviction, not your prompts. If the ceiling itself is 87%, the problem is your prompt template.

3 Step 2 — A worked example: 8 requests, 3 distinct prefixes

Take 8 Nexus requests. All share the 2100-token system prompt, then branch into one of three merchant-category profiles, then end in a unique per-transaction tail:

Req	Shared prefix bucket	Prefix len	Unique suffix
r1	sys+retail	2520	870
r2	sys+retail	2520	920
r3	sys+digital	2460	940
r4	sys+travel	2480	760
r5	sys+retail	2520	1180
r6	sys+digital	2460	990
r7	sys+retail	2520	810
r8	sys+digital	2460	1050

There are $N = 8$ requests but only **3** distinct prefix buckets (sys+retail, sys+digital, sys+travel).

The ceiling for this set

$$\text{max_hit_rate} = \frac{N - \text{unique_prefixes}}{N} = \frac{8 - 3}{8} = \frac{5}{8} = \mathbf{62.5\%}. \quad \checkmark$$

Five of the eight requests (r_2, r_5, r_7 on retail; r_6, r_8 on digital) arrive after their bucket is already warm, so their prefix KV is a hit. The three bucket-openers (r_1, r_3, r_4) are unavoidable misses.

4 Step 3 — Tokens recomputed, with and without sharing

The hit rate is nice, but the money is in *tokens of prefill not recomputed*. Without sharing, every request computes KV for its full prefix plus its suffix. With sharing, each distinct prefix is paid *once* and every request still pays its own suffix.

Count the prefill tokens

$$\begin{aligned} \text{no-share} &= \sum_i (\text{prefix}_i + \text{suffix}_i) = 27460 \text{ tokens,} \\ \text{shared} &= \underbrace{(2520 + 2460 + 2480)}_{\text{3 prefixes, once each}} + \underbrace{7520}_{\text{all suffixes}} = 7460 + 7520 = 14980 \text{ tokens,} \\ \text{saved} &= 27460 - 14980 = \mathbf{12480} \text{ tokens (45.4\% fewer).} \end{aligned}$$

The saving in bytes

At $\kappa = 320 \text{ KiB/token}$, those 12480 saved tokens are $12480 \times 320 \text{ KiB} = \mathbf{4.09 \text{ GB}}$ of KV that does *not* have to be recomputed for this batch of 8 requests — a 45% cut in prefill work. The system prompt alone, shared by all 8, is the bulk of it. The more requests share a prefix, the closer the saving approaches “pay the prefix exactly once, ever.”

5 Step 4 — The “system prompt paid once” headline

Push the idea to its limit. The Nexus 2100-token system prompt appears in *every* request. Its KV is:

$$L_{\text{sys}} \times \kappa = 2100 \times 320 \text{ KiB} = 688 \text{ MB},$$

computed *once* and reused for every request that follows.

Requests sharing it	Naive recompute	With RadixAttention
1000/hr	688 GB-equiv	688 MB once
47281/hr	32.5 TB-equiv	688 MB once ✓

Intuition

At 47281 requests per hour against one warm system-prompt node, the naive scheme would recompute 32.5 TB-equivalent of identical prefill KV — the same 688 MB, over and over. RadixAttention computes it once and serves 47281 hits off it. On measured Nexus traffic, prefix hits drop TTFT from 275 ms to 72 ms and lift peak throughput $\sim 2.3\times$, almost entirely from not recomputing shared-prompt KV.

6 Step 5 — The byte-stability footgun

The trie is keyed on token IDs, so the shared prefix must be *byte-for-byte identical* across requests. Anything that varies inside the prefix forks the trie path and destroys the hit rate.

Watch out

Putting "Current date: {date}" or a request ID or a UUID at the *top* of your system prompt is one of the most common production footguns. Every request then walks a different trie path and the hit rate collapses toward zero — the ceiling drops from 98.6% to nearly nothing while the engine looks blameless. The fix: keep all variable content *after* the stable prefix. Stable bytes first, per-request bytes last.

7 Step 6 — The production-scale ceiling

The same formula run on a real prompt log tells you the ceiling for your whole workload. Sampling $N = 10000$ prompts and bucketing by their first $L = 512$ characters:

Ceiling on a 10k-prompt log

$$\text{unique_prefixes} = 142, \quad \text{max_hit_rate} = \frac{10000 - 142}{10000} = \mathbf{98.6\%}.$$

Only 142 distinct prefixes among 10000 prompts means almost every request shares its opening with some earlier one. If the engine then reports 85%, the missing 13.6 points are eviction pressure — a capacity or scheduling problem, fixable with a bigger pool or sticky routing — and emphatically *not* a prompt-template problem. The ceiling tells you which world you are debugging in.

8 The argument, at a glance

#	Step	Result
1	Ceiling formula	$\text{max_hit} = (N - \text{unique})/N$
2	Worked set	8 reqs, 3 prefixes $\Rightarrow$ 62.5%
3	Tokens saved	27460 $\rightarrow$ 14980, save 12480 (4.09 GB)
4	System prompt	688 MB once vs 32.5 TB recomputed/hr
5	Footgun	variable content inside prefix kills the hit rate
6	Production ceiling	10k prompts, 142 unique $\Rightarrow$ 98.6%

8.1 Equation cheat-sheet

Hit-rate ceiling	$\text{max_hit_rate} = (N - \text{unique_prefixes})/N$
Tokens (no share)	$\sum_i (\text{prefix}_i + \text{suffix}_i)$
Tokens (shared)	$\sum_{\text{distinct prefixes}} \text{prefix} + \sum_i \text{suffix}_i$
KV saved	$(\text{tokens}_{\text{no-share}} - \text{tokens}_{\text{shared}}) \times \kappa$
Prefix KV	$L_{\text{prefix}} \times \kappa$ (paid once)
Diagnosis	ceiling – observed = eviction gap

Appendix A — Reference implementation

Every number above is produced by `m04_t03_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change the request set, the prefix buckets, or the log size to explore other workloads.

```

1  """
2  Reference implementation for: Module 04 - Topic 3
3  "RadixAttention - prefix sharing and the hit-rate ceiling."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  RadixAttention (Zheng et al., SGLang, 2024) keeps a trie of token sequences seen
9  so far. A new request reuses the KV of its longest already-cached prefix and only
10 computes KV for the new suffix. This script computes (a) the theoretical hit-rate
11 ceiling for a prompt log and (b) the KV bytes saved by paying a shared prefix once.
12 """
13 import math
14
15 # -----
16 # 0. KV per token (from Topic 1): FP16, GQA-8, 80 layers.
17 # -----
18 n_kv, d_head, n_layers, dtype = 8, 128, 80, 2
19 kv_per_token = 2 * n_kv * d_head * dtype * n_layers      # bytes/token (FP16)
20 print("== 0. KV per token (FP16, from Topic 1) ==")
21 print(f"kv_per_token = {kv_per_token} B = {kv_per_token/1024:.0f} KiB/token")
22
23 # -----
24 # 1. A small request set. Each request = (prefix_id, prefix_len, suffix_len).
25 #   'prefix_id' is the byte-stable shared prefix (e.g. system prompt) it walks.
26 # -----
27 # sys    : 2100-token Nexus system prompt (shared by all)
28 # retail/travel/digital : per-category profile appended after the system prompt
29 SYS = 2100
30 requests = [

```

```

31 # (label,          prefix_id,          prefix_len,  unique_suffix_len)
32 ("r1", "sys+retail",  SYS + 420, 870),
33 ("r2", "sys+retail",  SYS + 420, 920),
34 ("r3", "sys+digital", SYS + 360, 940),
35 ("r4", "sys+travel",  SYS + 380, 760),
36 ("r5", "sys+retail",  SYS + 420, 1180),
37 ("r6", "sys+digital", SYS + 360, 990),
38 ("r7", "sys+retail",  SYS + 420, 810),
39 ("r8", "sys+digital", SYS + 360, 1050),
40 ]
41 N = len(requests)
42
43 # -----
44 # 2. Hit-rate ceiling: max_hit_rate = (N - unique_prefixes) / N.
45 #   A request is a "hit" if some EARLIER request shares its prefix bucket.
46 # -----
47 prefix_ids = [r[1] for r in requests]
48 unique_pref = len(set(prefix_ids))
49 max_hit_rate = (N - unique_pref) / N
50 print("\n== 1. Hit-rate ceiling for this 8-request set ==")
51 print(f"N               = {N} requests")
52 print(f"unique_prefixes    = {unique_pref} ({sorted(set(prefix_ids))})")
53 print(f"max_hit_rate         = (N - unique_prefixes)/N = ({N} - {unique_pref})/{N} = {max_hit_rate*100:.1f}%")
54
55 # -----
56 # 3. KV computed WITHOUT sharing vs WITH sharing.
57 #   Without: every request computes prefix_len + suffix_len of KV.
58 #   With    : each distinct prefix bucket pays its prefix_len ONCE; every
59 #             request still pays its own unique suffix.
60 # -----
61 tokens_no_share = sum(r[2] + r[3] for r in requests)
62
63 # pay each distinct prefix once (use its prefix_len) + all suffixes
64 seen = {}
65 prefix_once = 0
66 for _, pid, plen, _ in requests:
67     if pid not in seen:
68         seen[pid] = plen
69         prefix_once += plen
70 suffix_total = sum(r[3] for r in requests)
71 tokens_share = prefix_once + suffix_total
72
73 print("\n== 2. KV tokens computed: no-share vs RadixAttention ==")
74 print(f"no-share tokens      = sum(prefix+suffix) = {tokens_no_share}")
75 print(f"shared tokens        = (distinct prefixes paid once) + suffixes")
76 print(f"                     = {prefix_once} + {suffix_total} = {tokens_share}")
77 saved = tokens_no_share - tokens_share
78 print(f"tokens saved         = {saved} ({saved/tokens_no_share*100:.1f}% fewer prefill tokens)")
79
80 # -----
81 # 4. Translate token savings into KV bytes and prefill work.
82 # -----
83 print("\n== 3. The savings in bytes ==")
84 print(f"KV recomputed without sharing = {tokens_no_share*kv_per_token/1e9:.3f} GB-equiv of KV writes")
85 print(f"KV recomputed with sharing    = {tokens_share*kv_per_token/1e9:.3f} GB-equiv")
86 print(f"saved                         = {saved*kv_per_token/1e9:.3f} GB-equiv of prefill KV")
87
88 # -----
89 # 5. The 'pay the system prompt once' headline.
90 #   System prompt = 2100 tokens, shared by 100% of traffic.
91 # -----
92 print("\n== 4. System prompt paid once vs per-request ==")
93 sys_kv = SYS * kv_per_token

```

```

94 print(f"system prompt KV = {SYS} tok x {kv_per_token} B = {sys_kv/1e6:.1f} MB (computed ONCE, reused)")
95 for reqs_per_hour in [1000, 47281]:
96     naive = reqs_per_hour * sys_kv
97     print(f" at {reqs_per_hour:>6} reqs/hr: naive recompute = {naive/1e9:.1f} GB-equiv of "
98           f"wasted prefill; with RadixAttention ~ {sys_kv/1e6:.1f} MB once")
99
100 # -----
101 # 6. Production-scale ceiling (the 98.6% headline from Section 8).
102 # -----
103 print("\n== 5. Production-scale ceiling (sampled prompt log) ==")
104 Nlog, uniq = 10000, 142
105 print(f"N={Nlog} prompts, unique_prefixes at L=512 chars = {uniq}")
106 print(f"max_hit_rate = ({Nlog} - {uniq})/{Nlog} = {(Nlog-uniq)/Nlog*100:.1f}%")

```

— END OF DOCUMENT —